

패턴 인식을 위한 조밀 연상 기억

Dense Associative Memory for Pattern Recognition

Dmitry Krotov, John J. Hopfield

문서 성격 원문 순서 · 수치 · 한정어를 보존한 한국어 구조 보존형 의역본이다. 본문 뒤의 원문 보존 부록은 원 논문의 equation, table, figure, caption 을 검증 가능하게 남긴다. 인용과 재현에는 반드시 고정 원문을 사용한다.

고정 버전 arXiv:1606.01164v2

배포 상태 internal-only — arXiv/proceedings rights; translated derivative and source plate are internal-only

Contents

초록 (Abstract)	2
1. 서론 (Introduction)	2
2. 큰 용량의 연상 기억 (Associative Memory with Large Capacity)	2
3. XOR 사례 (The Case of XOR)	3
4. MNIST 패턴 인식 (An Example: MNIST)	3
5. 한 개 hidden layer network 와의관계	3
6. 논의와 결론 (Discussion and Conclusions)	3
원문 구조 안내	4
번역 · 검증 고지	5
원문 구조 · 그림 보존 부록	6

초록 (Abstract)

이 논문은 network 의 neuron 수보다 훨씬 많은 pattern 을 저장하고 안정적으로 검색할 수 있는 associative memory 를 연구한다. 저자들은 **dense associative memory** 와 deep learning 에서 흔히 쓰이는 feedforward network 사이의 단순한 duality 를 제시한다. associative-memory 쪽에서는 feature-matching regime 과 prototype regime 사이를 매끄럽게 잇는 model family 가 만들어진다. deep-learning 쪽에서 이 family 는 한 개 hidden layer 와 서로 다른 activation function 을 갖는 network 에 대응한다.

activation family 에는 logistic, rectified linear unit(ReLU), 더 높은 차수의 rectified polynomial 이 포함된다. 이 duality 를 통해 energy-based memory 의 직관으로 unusual activation 을 쓰는 network 의 계산 성질을 분석할 수 있다. XOR 과 MNIST handwritten-digit recognition 을 예로 들어 dense memory 의 용도를 보인다.

1. 서론 (Introduction)

pattern recognition 과 associative memory 는 연결되어 있다. image 와 label 을 하나의 memory vector 로 저장하면, label 이 없는 image 는 불완전한 cue 이고 올바른 label 을 찾는 일은 memory completion 이 된다. 표준 Hopfield memory 는 저장 pattern 수가 neuron 수보다 훨씬 작을 때 잘 작동한다. 실제 classification 에서는 pixel 수보다 훨씬 많은 example 을 구별해야 하므로 이 capacity 가 병목이다.

표준 energy 의 neuron interaction 은 quadratic 이다. 논문은 interaction order 를 높인 energy 를 사용하면 capacity 를 크게 늘릴 수 있다고 설명한다. 동시에 이 higher-order memory 를 feedforward neural network 로 다시 표현하면, higher-order rectified activation 이 자연스럽게 나온다. 따라서 memory capacity 와 activation design 을 하나의 수학적 구조에서 다룰 수 있다.

2. 큰 용량의 연상 기억 (Associative Memory with Large Capacity)

(N) 개의 binary neuron state 를 σ , 저장할 (K) 개의 memory pattern 을 ξ^μ 라고 하자. 표준 Hopfield model 은 overlap 의 quadratic function 으로 energy 를 만든다. dense associative memory 는 각 pattern 과 현재 state 의 overlap 에 더 높은 차수의 function (F) 를 적용한다.

$$E(\sigma) = - \sum_{\mu=1}^K F \left(\sum_{i=1}^N \xi_i^\mu \sigma_i \right).$$

한 neuron 을 update 할 때에는 그 neuron 을 (+1) 또는 (-1) 로 두었을 때의 energy 차이를 계산하고 energy 가 낮아지는 방향을 택한다. (F(x)=x^2) 이면 표준 quadratic memory 에 가깝다. (F) 의 차수가 (n) 인 polynomial 이면 high-order interaction 이 생기고, 적절한 조건에서 저장 capacity 는 (N) 보다 훨씬 빠르게 증가한다. 논문이 강조하는 것은 높은 차수가 단지 parameter 수를 늘리는 것이 아니라, 잘못된 pattern 의 cross-talk 를 억제해 많은 memory 를 안정적으로 분리한다는 점이다.

입력과 저장 pattern 의 overlap 이 양수일 때만 크게 반응하도록 rectified polynomial 을 사용한다. 이 비선형성은 현재 cue 와 충분히 맞는 memory 만 update 에 강하게 기여하게 한다. 차수가 커질수록 가장 잘 맞는 prototype 이 지배하고, 낮은 차수에서는 여러 example 의 공통 feature 가 함께 기여한다.

3. XOR 사례 (The Case of XOR)

XOR은 선형 classifier로 풀 수 없는 가장 작은 예다. dense associative memory는 input bit와 output label을 결합한 pattern을 저장하고, label component가 비어 있는 cue에서 energy descent로 label을 복원한다. 표준 quadratic interaction에서는 서로 간섭하는 pattern이 생기지만 higher-order term은 올바른 basin을 형성한다.

저자들은 interaction power와 threshold를 바꾸면서 memory landscape가 어떻게 달라지는지 보인다. 낮은 power에서는 개별 example보다 shared direction에 반응하는 feature regime이 나타나고, 높은 power에서는 training example과 가까운 prototype basin이 강해진다. XOR은 이 두 regime의 차이를 시각적으로 설명하기 위한 통제 예제다.

4. MNIST 패턴 인식 (An Example: MNIST)

MNIST 실험에서는 handwritten digit image와 label을 joint vector로 구성한다. query시 image pixel을 주고 label unit을 update하여 digit을 분류한다. dense memory는 많은 training pattern을 저장하면서도 cross-talk를 제한해야 한다. polynomial order, hidden unit 수, regularization에 따라 test error와 learned representation이 달라진다.

feature-matching regime의 hidden weights는 stroke나 edge처럼 여러 image가 공유하는 부분을 표현한다. prototype regime에서는 특정 digit example 전체와 유사한 template가 나타난다. 논문은 높은 차수 activation이언제나 더 낫다고 말하지 않는다. data와 parameterization에 따라 feature generalization과 prototype recall 사이 trade-off가 있다.

5. 한 개 hidden layer network와의 관계

energy를 한 neuron 또는 label component에 관해 비교하면, 저장 pattern과 input의 dot product를 계산한 뒤 nonlinear function을 적용하고 output 방향으로 합하는 식이 얻어진다. 이는 한 개 hidden layer feedforward network와 같은 계산이다.

$$h_{\mu} = f(\xi^{\mu} \cdot \mathbf{x} - \theta_{\mu}), \quad y_i = \sum_{\mu} \xi_i^{\mu} h_{\mu}.$$

memory vector는 input-to-hidden weight와 hidden-to-output weight 양쪽에 나타난다. (f)는 energy function (F)의 차분 또는 derivative에 해당한다. quadratic memory는 비교적 낮은 차수 activation에, higher-order memory는 rectified polynomial에 대응한다. logistic과 ReLU도 이 family의 특징 선택으로 해석할 수 있다.

이 duality는 associative memory의 attractor와 basin 개념을 feedforward inference에 옮긴다. 반대로 neural network training을 memory pattern을 학습하는 과정으로 볼 수 있다. weight tying을 풀어 일반 feedforward network로 확장할 수도 있지만, 그 경우 정확한 energy interpretation은 약해진다.

6. 논의와 결론 (Discussion and Conclusions)

dense associative memory는 높은 차수 interaction으로 neuron 수보다 많은 pattern을 저장하는 수학적 가능성을 보인다. 하지만 asymptotic capacity와 실제 finite network의 usable capacity는 같지 않다. noise, correlated data, learning된 memory vector, inference cost가

성능을 제한한다. 모든 pattern 을 직접 저장하면 parameter 와 compute 가 pattern 수에 따라증가한다.

MNIST 결과는 higher rectified polynomial 이 실제 pattern recognition 에 쓸수 있음을 보이는 예이지, 현대 deep architecture 전반에 대한 우월성을 입증하는 결과는 아니다. 연구는 one-hidden-layer duality 와 energy landscape 를 중심으로 하며, continual write, eviction, sleep-time consolidation, distributed serving 을 직접 다루지 않는다.

이 논문의 장기기억 관련 핵심은 capacity 를 energy 와 activation order 로 분석할 수 있다는 데 있다. 무엇을 저장하고 언제 삭제할지, 순차 학습 중 interference 를 어떻게 막을지는 별도의 문제로 남는다.

원문 구조 안내

뒤의 원문 보존 부록에는 binary-state energy, update rule, capacity 조건, XOR energy landscape, MNIST 설정, neural-network duality 의 모든 equation 과 Figure 1 – 5 가 v2 PDF 그대로 포함된다.

번역·검증 고지

이 문서는 분석이나 평가를 추가하지 않는 것을 원칙으로 하며, 원문의 주장 강도와 수치를 유지하도록 작성되었다. 한국어 초안 작성과 구조 검수에는 Codex GPT-5.6 을 사용했고, 고정 source hash·원문 보존 부록·역대조 표를 통해 검증했다. 불일치가 있을 때에는 원문이 우선한다.

원문 구조·그림 보존 부록

아래 부록은 고정 원문의 equation, table, figure, caption 과 section 배치를 대조하기 위한 검증판이다. arXiv/proceedings rights; translated derivative and source plate are internal-only

Dense Associative Memory for Pattern Recognition

Dmitry Krotov

Simons Center for Systems Biology
Institute for Advanced Study
Princeton, USA
krotov@ias.edu

John J. Hopfield

Princeton Neuroscience Institute
Princeton University
Princeton, USA
hopfield@princeton.edu

Abstract

A model of associative memory is studied, which stores and reliably retrieves many more patterns than the number of neurons in the network. We propose a simple duality between this dense associative memory and neural networks commonly used in deep learning. On the associative memory side of this duality, a family of models that smoothly interpolates between two limiting cases can be constructed. One limit is referred to as the feature-matching mode of pattern recognition, and the other one as the prototype regime. On the deep learning side of the duality, this family corresponds to feedforward neural networks with one hidden layer and various activation functions, which transmit the activities of the visible neurons to the hidden layer. This family of activation functions includes logistics, rectified linear units, and rectified polynomials of higher degrees. The proposed duality makes it possible to apply energy-based intuition from associative memory to analyze computational properties of neural networks with unusual activation functions – the higher rectified polynomials which until now have not been used in deep learning. The utility of the dense memories is illustrated for two test cases: the logical gate XOR and the recognition of handwritten digits from the MNIST data set.

1 Introduction

Pattern recognition and models of associative memory [1] are closely related. Consider image classification as an example of pattern recognition. In this problem, the network is presented with an image and the task is to label the image. In the case of associative memory the network stores a set of memory vectors. In a typical query the network is presented with an incomplete pattern resembling, but not identical to, one of the stored memories and the task is to recover the full memory. Pixel intensities of the image can be combined together with the label of that image into one vector [2], which will serve as a memory for the associative memory. Then the image itself can be thought of as a partial memory cue. The task of identifying an appropriate label is a subpart of the associative memory reconstruction. There is a limitation in using this idea to do pattern recognition. The standard model of associative memory works well in the limit when the number of stored patterns is much smaller than the number of neurons [1], or equivalently the number of pixels in an image. In order to do pattern recognition with small error rate one would need to store many more memories than the typical number of pixels in the presented images. This is a serious problem. It can be solved by modifying the standard energy function of associative memory, quadratic in interactions between the neurons, by including in it higher order interactions. By properly designing the energy function (or Hamiltonian) for these models with higher order interactions one can store and reliably retrieve many more memories than the number of neurons in the network.

Deep neural networks have proven to be useful for a broad range of problems in machine learning including image classification, speech recognition, object detection, etc. These models are composed of several layers of neurons, so that the output of one layer serves as the input to the next layer. Each

neuron calculates a weighted sum of the inputs and passes the result through a non-linear activation function. Traditionally, deep neural networks used activation functions such as hyperbolic tangents or logistics. Learning the weights in such networks, using a backpropagation algorithm, faced serious problems in the 1980s and 1990s. These issues were largely resolved by introducing unsupervised pre-training, which made it possible to initialize the weights in such a way that the subsequent backpropagation could only gently move boundaries between the classes without destroying the feature detectors [3, 4]. More recently, it was realized that the use of rectified linear units (ReLU) instead of the logistic functions speeds up learning and improves generalization [5, 6, 7]. Rectified linear functions are usually interpreted as firing rates of biological neurons. These rates are equal to zero if the input is below a certain threshold and linearly grow with the input if it is above the threshold. To mimic biology the output should be small or zero if the input is below the threshold, but it is much less clear what the behavior of the activation function should be for inputs exceeding the threshold. Should it grow linearly, sub-linearly, or faster than linearly? How does this choice affect the computational properties of the neural network? Are there other functions that would work even better than the rectified linear units? These questions to the best of our knowledge remain open.

This paper examines these questions through the lens of associative memory. We start by discussing a family of models of associative memory with large capacity. These models use higher order (higher than quadratic) interactions between the neurons in the energy function. The associative memory description is then mapped onto a neural network with one hidden layer and an unusual activation function, related to the Hamiltonian. We show that by varying the power of interaction vertex in the energy function (or equivalently by changing the activation function of the neural network) one can force the model to learn representations of the data either in terms of features or in terms of prototypes.

2 Associative memory with large capacity

The standard model of associative memory [1] uses a system of N binary neurons, with values ± 1 . A configuration of all the neurons is denoted by a vector σ_i . The model stores K memories, denoted by ξ_i^μ , which for the moment are also assumed to be binary. The model is defined by an energy function, which is given by

$$E = -\frac{1}{2} \sum_{i,j=1}^N \sigma_i T_{ij} \sigma_j, \quad T_{ij} = \sum_{\mu=1}^K \xi_i^\mu \xi_j^\mu, \quad (1)$$

and a dynamical update rule that decreases the energy at every update. The basic problem is the following: when presented with a new pattern the network should respond with a stored memory which most closely resembles the input.

There has been a large amount of work in the community of statistical physicists investigating the capacity of this model, which is the maximal number of memories that the network can store and reliably retrieve. It has been demonstrated [1, 8, 9] that in case of random memories this maximal value is of the order of $K^{max} \approx 0.14N$. If one tries to store more patterns, several neighboring memories in the configuration space will merge together producing a ground state of the Hamiltonian (1), which has nothing to do with any of the stored memories. By modifying the Hamiltonian (1) in a way that removes second order correlations between the stored memories, it is possible [10] to improve the capacity to $K^{max} = N$.

The mathematical reason why the model (1) gets confused when many memories are stored is that several memories produce contributions to the energy which are of the same order. In other words the energy decreases too slowly as the pattern approaches a memory in the configuration space. In order to take care of this problem, consider a modification of the standard energy

$$E = - \sum_{\mu=1}^K F(\xi_i^\mu \sigma_i) \quad (2)$$

In this formula $F(x)$ is some smooth function (summation over index i is assumed). The computational capabilities of the model will be illustrated for two cases. First, when $F(x) = x^n$ (n is an integer number), which is referred to as a polynomial energy function. Second, when $F(x)$ is a

rectified polynomial energy function

$$F(x) = \begin{cases} x^n, & x \geq 0 \\ 0, & x < 0 \end{cases} \quad (3)$$

In the case of the polynomial function with $n = 2$ the network reduces to the standard model of associative memory [1]. If $n > 2$ each term in (2) becomes sharper compared to the $n = 2$ case, thus more memories can be packed into the same configuration space before cross-talk intervenes.

Having defined the energy function one can derive an iterative update rule that leads to decrease of the energy. We use asynchronous updates flipping one unit at a time. The update rule is:

$$\sigma_i^{(t+1)} = \text{Sign} \left[\sum_{\mu=1}^K \left(F \left(\xi_i^\mu + \sum_{j \neq i} \xi_j^\mu \sigma_j^{(t)} \right) - F \left(-\xi_i^\mu + \sum_{j \neq i} \xi_j^\mu \sigma_j^{(t)} \right) \right) \right], \quad (4)$$

The argument of the sign function is the difference of two energies. One, for the configuration with all but the i -th units clumped to their current states and the i -th unit in the “off” state. The other one for a similar configuration, but with the i -th unit in the “on” state. This rule means that the system updates a unit, given the states of the rest of the network, in such a way that the energy of the entire configuration decreases. For the case of polynomial energy function a very similar family of models was considered in [11, 12, 13, 14, 15, 16]. The update rule in those models was based on the induced magnetic fields, however, and not on the difference of energies. The two are slightly different due to the presence of self-coupling terms. Throughout this paper we use energy-based update rules.

How many memories can model (4) store and reliably retrieve? Consider the case of random patterns, so that each element of the memories is equal to ± 1 with equal probability. Imagine that the system is initialized in a state equal to one of the memories (pattern number μ). One can derive a stability criterion, i.e. the upper bound on the number of memories such that the network stays in that initial state. Define the energy difference between the initial state and the state with spin i flipped

$$\Delta E = \sum_{\nu=1}^K \left(\xi_i^\nu \xi_i^\mu + \sum_{j \neq i} \xi_j^\nu \xi_j^\mu \right)^n - \sum_{\nu=1}^K \left(-\xi_i^\nu \xi_i^\mu + \sum_{j \neq i} \xi_j^\nu \xi_j^\mu \right)^n,$$

where the polynomial energy function is used. This quantity has a mean $\langle \Delta E \rangle = N^n - (N-2)^n \approx 2nN^{n-1}$, which comes from the term with $\nu = \mu$, and a variance (in the limit of large N)

$$\Sigma^2 = \Omega_n (K-1) N^{n-1}, \quad \text{where} \quad \Omega_n = 4n^2 (2n-3)!!$$

The i -th bit becomes unstable when the magnitude of the fluctuation exceeds the energy gap $\langle \Delta E \rangle$ and the sign of the fluctuation is opposite to the sign of the energy gap. Thus the probability that the state of a single neuron is unstable (in the limit when both N and K are large, so that the noise is effectively gaussian) is equal to

$$P_{\text{error}} = \int_{\langle \Delta E \rangle}^{\infty} \frac{dx}{\sqrt{2\pi\Sigma^2}} e^{-\frac{x^2}{2\Sigma^2}} \approx \sqrt{\frac{(2n-3)!!}{2\pi}} \frac{K}{N^{n-1}} e^{-\frac{K^2 N^{n-1}}{2K(2n-3)!!}}$$

Requiring that this probability is less than a small value, say 0.5%, one can find the upper limit on the number of patterns that the network can store

$$K^{\text{max}} = \alpha_n N^{n-1}, \quad (5)$$

where α_n is a numerical constant, which depends on the (arbitrary) threshold 0.5%. The case $n = 2$ corresponds to the standard model of associative memory and gives the well known result $K = 0.14N$. For the perfect recovery of a memory ($P_{\text{error}} < 1/N$) one obtains

$$K_{\text{no errors}}^{\text{max}} \approx \frac{1}{2(2n-3)!! \ln(N)} N^{n-1} \quad (6)$$

For higher powers n the capacity rapidly grows with N in a non-linear way, allowing the network to store and reliably retrieve many more patterns than the number of neurons that it has, in accord¹ with [13, 14, 15, 16]. This non-linear scaling relationship between the capacity and the size of the network is the phenomenon that we exploit.

¹The n -dependent coefficient in (6) depends on the exact form of the Hamiltonian and the update rule. References [13, 14, 15] do not allow repeated indices in the products over neurons in the energy function, therefore obtain a different coefficient. In [16] the Hamiltonian coincides with ours, but the update rule is different, which, however, results in exactly the same coefficient as in (6).

We study a family of models of this kind as a function of n . At small n many terms contribute to the sum over μ in (2) approximately equally. In the limit $n \rightarrow \infty$ the dominant contribution to the sum comes from a single memory, which has the largest overlap with the input. It turns out that optimal computation occurs in the intermediate range.

3 The case of XOR

The case of XOR is elementary, yet instructive. It is presented here for three reasons. First, it illustrates the construction (2) in this simplest case. Second, it shows that as n increases, the computational capabilities of the network also increase. Third, it provides the simplest example of a situation in which the number of memories is larger than the number of neurons, yet the network works reliably.

The problem is the following: given two inputs x and y produce an output z such that the truth table

x	y	z
-1	-1	-1
-1	1	1
1	-1	1
1	1	-1

is satisfied. We will treat this task as an associative memory problem and will simply embed the four examples of the input-output triplets x, y, z in the memory. Therefore the network has $N = 3$ identical units: two of which will be used for the inputs and one for the output, and $K = 4$ memories ξ_i^μ , which are the four lines of the truth table. Thus, the energy (2) is equal to

$$E_n(x, y, z) = -(-x - y - z)^n - (-x + y + z)^n - (x - y + z)^n - (x + y - z)^n, \quad (7)$$

where the energy function is chosen to be a polynomial of degree n . For odd n , energy (7) is an odd function of each of its arguments, $E_n(x, y, -z) = -E_n(x, y, z)$. For even n , it is an even function. For $n = 1$ it is equal to zero. Thus, if evaluated on the corners of the cube $x, y, z = \pm 1$, it reduces to

$$E_n(x, y, z) = \begin{cases} 0, & n = 1 \\ C_n, & n = 2, 4, 6, \dots \\ C_n xyz, & n = 3, 5, 7, \dots, \end{cases} \quad (8)$$

where coefficients C_n denote numerical constants.

In order to solve the XOR problem one can present to the network an “incomplete pattern” of inputs (x, y) and let the output z adjust to minimize the energy of the three-spin configuration, while holding the inputs fixed. The network clearly cannot solve this problem for $n = 1$ and $n = 2$, since the energy does not depend on the spin configuration. The case $n = 2$ is the standard model of associative memory. It can also be thought of as a linear perceptron, and the inability to solve this problem represents the well known statement [17] that linear perceptrons cannot compute XOR without hidden neurons. The case of odd $n \geq 3$ provides an interesting solution. Given two inputs, x and y , one can choose the output z that minimizes the energy. This leads to the update rule

$$z = \text{Sign}[E_n(x, y, -1) - E_n(x, y, +1)] = \text{Sign}[-xy]$$

Thus, in this simple case the network is capable of solving the problem for higher odd values of n , while it cannot do so for $n = 1$ and $n = 2$. In case of rectified polynomials, a similar construction solves the problem for any $n \geq 2$. The network works well in spite of the fact that $K > N$.

4 An example of a pattern recognition problem, the case of MNIST

The MNIST data set is a collection of handwritten digits, which has 60000 training examples and 10000 test images. The goal is to classify the digits into 10 classes. The visible neurons, one for each pixel, are combined together with 10 classification neurons in one vector that defines the state of the network. The visible part of this vector is treated as an “incomplete” pattern and the associative memory is allowed to calculate a completion of that pattern, which is the label of the image.

Dense associative memory (2) is a recurrent network in which every neuron can be updated multiple times. For the purposes of digit classification, however, this model will be used in a very limited

capacity, allowing it to perform only one update of the classification neurons. The network is initialized in the state when the visible units v_i are clamped to the intensities of a given image and the classification neurons are in the off state $x_\alpha = -1$ (see Fig.1A). The network is allowed to make one update of the classification neurons, while keeping the visible units clamped, to produce the output c_α . The update rule is similar to (4) except that the sign is replaced by the continuous function $g(x) = \tanh(x)$

$$c_\alpha = g \left[\beta \sum_{\mu=1}^K \left(F \left(-\xi_\alpha^\mu x_\alpha + \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^N \xi_i^\mu v_i \right) - F \left(\xi_\alpha^\mu x_\alpha + \sum_{\gamma \neq \alpha} \xi_\gamma^\mu x_\gamma + \sum_{i=1}^N \xi_i^\mu v_i \right) \right) \right], \quad (9)$$

where parameter β regulates the slope of $g(x)$. The proposed digit class is given by the number of a classification neuron producing the maximal output. Throughout this section the rectified polynomials (3) are used as functions F . To learn effective memories for use in pattern classification, an objective function is defined (see Appendix A in Supplemental), which penalizes the discrepancy

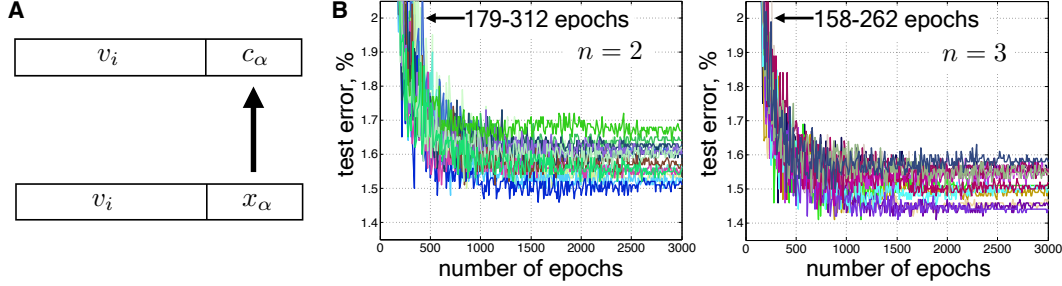


Figure 1: (A) The network has $N = 28 \times 28 = 784$ visible neurons and $N_c = 10$ classification neurons. The visible units are clamped to intensities of pixels (which is mapped on the segment $[-1, 1]$), while the classification neurons are initialized in the state x_α and then updated once to the state c_α . (B) Behavior of the error on the test set as training progresses. Each curve corresponds to a different combination of hyperparameters from the optimal window, which was determined on the validation set. The arrows show the first time when the error falls below a 2% threshold. All models have $K = 2000$ memories (hidden units).

between the output c_α and the target output. This objective function is then minimized using a backpropagation algorithm. The learning starts with random memories drawn from a Gaussian distribution. The backpropagation algorithm then finds a collection of K memories $\xi_{i,\alpha}^\mu$, which minimize the classification error on the training set. The memories are normalized to stay within the $-1 \leq \xi_{i,\alpha}^\mu \leq 1$ range, absorbing their overall scale into the definition of the parameter β .

The performance of the proposed classification framework is studied as a function of the power n . The next section shows that a rectified polynomial of power n in the energy function is equivalent to the rectified polynomial of power $n - 1$ used as an activation function in a feedforward neural network with one hidden layer of neurons. Currently, the most common choice of activation functions for training deep neural networks is the ReLU, which in our language corresponds to $n = 2$ for the energy function. Although not currently used to train deep networks, the case $n = 3$ would correspond to a rectified parabola as an activation function. We start by comparing the performances of the dense memories in these two cases.

The performance of the network depends on n and on the remaining hyperparameters, thus the hyperparameters should be optimized for each value of n . In order to test the variability of performances for various choices of hyperparameters at a given n , a window of hyperparameters for which the network works well on the validation set (see the Appendix A in Supplemental) was determined. Then many networks were trained for various choices of the hyperparameters from this window to evaluate the performance on the test set. The test errors as training progresses are shown in Fig.1B. While there is substantial variability among these samples, on average the cluster of trajectories for $n = 3$ achieves better results on the test set than that for $n = 2$. These error rates should be compared with error rates for backpropagation alone without the use of generative pretraining, various kinds of regularizations (for example dropout) or adversarial training, all of which could be added to our construction if necessary. In this class of models the best published results are all² in the 1.6% range [18], see also controls in [19, 20]. This agrees with our results for $n = 2$. The $n = 3$ case does slightly better than that as is clear from Fig.1B, with all the samples performing better than 1.6%.

²Although there are better results on pixel permutation invariant task, see for example [19, 20, 21, 22].

Higher rectified polynomials are also faster in training compared to ReLU. For the $n = 2$ case, the error crosses the 2% threshold for the first time during training in the range of 179-312 epochs. For the $n = 3$ case, this happens earlier on average, between 158-262 epochs. For higher powers n this speed-up is larger. This is not a huge effect for a small dataset such as MNIST. However, this speed-up might be very helpful for training large networks on large datasets, such as ImageNet. A similar effect was reported earlier for the transition between saturating units, such as logistics or hyperbolic tangents, to ReLU [7]. In our family of models that result corresponds to moving from $n = 1$ to $n = 2$.

Feature to prototype transition

How does the computation performed by the neural network change as n varies? There are two extreme classes of theories of pattern recognition: feature-matching and formation of a prototype. According to the former, an input is decomposed into a set of features, which are compared with those stored in the memory. The subset of the stored features activated by the presented input is then interpreted as an object. One object has many features; features can also appear in more than one object. The prototype theory provides an alternative approach, in which objects are recognized as a whole. The prototypes do not necessarily match the object exactly, but rather are blurred abstract

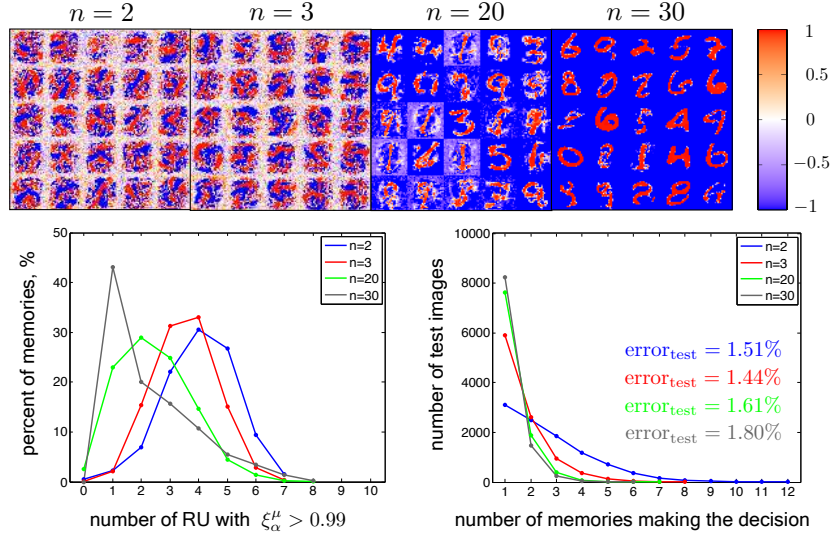


Figure 2: We show 25 randomly selected memories (feature detectors) for four networks, which use rectified polynomials of degrees $n = 2, 3, 20, 30$ as the energy function. The magnitude of a memory element corresponding to each pixel is plotted in the location of that pixel, the color bar explains the color code. The histograms at the bottom are explained in the text. The error rates refer to the particular four samples used in this figure. RU stands for recognition unit.

representations which include all the features that an object has. We argue that the computational models proposed here describe feature-matching mode of pattern recognition for small n and the prototype regime for large n . This can be anticipated from the sharpness of contributions that each memory makes to the total energy (2). For large n the function $F(x)$ peaks much more sharply around each memory compared to the case of small n . Thus, at large n all the information about a digit must be written in only one memory, while at small n this information can be distributed among several memories. In the case of intermediate n some learned memories behave like features while others behave like prototypes. These two classes of memories work together to model the data in an efficient way.

The feature to prototype transition is clearly seen in memories shown in Fig.2. For $n = 2$ or 3 each memory does not look like a digit, but resembles a pattern of activity that might be useful for recognizing several different digits. For $n = 20$ many of the memories can be recognized as digits, which are surrounded by white margins representing elements of memories having approximately zero values. These margins describe the variability of thicknesses of lines of different training examples and mathematically mean that the energy (2) does not depend on whether the corresponding pixel is on or off. For $n = 30$ most of the memories represent prototypes of whole digits or large portions of digits, with a small admixture of feature memories that do not resemble any digit.

The feature to prototype transition can be visualized by showing the feature detectors in situations when there is a natural ordering of pixels. Such ordering exists in images, for example. In general situations, however, there is no preferred permutation of visible neurons that would reveal this structure (*e.g.* in the case of genomic data). It is therefore useful to develop a measure that permits a distinction to be made between features and prototypes in the absence of such visual space. Towards the end of training most of the recognition connections ξ_α^μ are approximately equal to ± 1 . One can choose an arbitrary cutoff, and count the number of recognition connections that are in the “on” state ($\xi_\alpha^\mu = +1$) for each memory. The distribution function of this number is shown on the left histogram in Fig.2. Intuitively, this quantity corresponds to the number of different digit classes that a particular memory votes for. At small n , most of the memories vote for three to five different digit classes, a behavior characteristic of features. As n increases, each memory specializes and votes for only a single class. In the case $n = 30$, for example, more than 40% of memories vote for only one class, a behavior characteristic of prototypes. A second way to see the feature to prototype transition is to look at the number of memories which make large contributions to the classification decision (right histogram in Fig.2). For each test image one can find the memory that makes the largest contribution to the energy gap, which is the sum over μ in (9). Then one can count the number of memories that contribute to the gap by more than 0.9 of this largest contribution. For small n , there are many memories that satisfy this criterion and the distribution function has a long tail. In this regime several memories are cooperating with each other to make a classification decision. For $n = 30$, however, more than 8000 of 10000 test images do not have a single other memory that would make a contribution comparable with the largest one. This result is not sensitive to the arbitrary choice (0.9) of the cutoff. Interestingly, the performance remains competitive even for very large $n \approx 20$ (see Fig.2) in spite of the fact that these networks are doing a very different kind of computation compared with that at small n .

5 Relationship to a neural network with one hidden layer

In this section we derive a simple duality between the dense associative memory and a feedforward neural network with one layer of hidden neurons. In other words, we show that the same computational model has two very different descriptions: one in terms of associative memory, the other one in terms

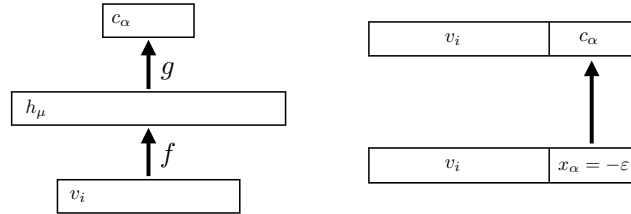


Figure 3: On the left a feedforward neural network with one layer of hidden neurons. The states of the visible units are transformed to the hidden neurons using a non-linear function f , the states of the hidden units are transformed to the output layer using a non-linear function g . On the right the model of dense associative memory with one step update (9). The two models are equivalent.

of a network with one layer of hidden units. Using this correspondence one can transform the family of dense memories, constructed for different values of power n , to the language of models used in deep learning. The resulting neural networks are guaranteed to inherit computational properties of the dense memories such as the feature to prototype transition.

The construction is very similar to (9), except that the classification neurons are initialized in the state when all of them are equal to $-\varepsilon$, see Fig.3. In the limit $\varepsilon \rightarrow 0$ one can expand the function F in (9) so that the dominant contribution comes from the term linear in ε . Then

$$c_\alpha \approx g \left[\beta \sum_{\mu=1}^K F' \left(\sum_{i=1}^N \xi_i^\mu v_i \right) (-2\xi_\alpha^\mu x_\alpha) \right] = g \left[\sum_{\mu=1}^K \xi_\alpha^\mu F'(\xi_i^\mu v_i) \right] = g \left[\sum_{\mu=1}^K \xi_\alpha^\mu f(\xi_i^\mu v_i) \right], \quad (10)$$

where the parameter β is set to $\beta = 1/(2\varepsilon)$ (summation over the visible index i is assumed). Thus, the model of associative memory with one step update is equivalent to a conventional feedforward neural network with one hidden layer provided that the activation function from the visible layer to the hidden layer is equal to the derivative of the energy function

$$f(x) = F'(x) \quad (11)$$

The visible part of each memory serves as an incoming weight to the hidden layer, and the recognition part of the memory serves as an outgoing weight from the hidden layer. The expansion used in (10)

is justified by a condition $\sum_{i=1}^N \xi_i^\mu v_i \gg \sum_{\alpha=1}^{N_c} \xi_\alpha^\mu x_\alpha$, which is satisfied for most common problems, and is simply a statement that labels contain far less information than the data itself.

From the point of view of associative memory, the dominant contribution shaping the basins of attraction comes from the low energy states. Therefore mathematically it is determined by the asymptotics of the activation function $f(x)$, or the energy function $F(x)$, at $x \rightarrow \infty$. Thus different activation functions having similar asymptotics at $x \rightarrow \infty$ should fall into the same universality class and should have similar computational properties. In the table below we list some common activation

activation function	energy function	n
$f(x) = \tanh(x)$	$F(x) = \ln(\cosh(x)) \approx x$, at $x \rightarrow \infty$	1
$f(x) = \text{logistic function}$	$F(x) = \ln(1 + e^x) \approx x$, at $x \rightarrow \infty$	1
$f(x) = \text{ReLU}$	$F(x) \sim x^2$, at $x \rightarrow \infty$	2
$f(x) = \text{ReP}_{n-1}$	$F(x) = \text{ReP}_n$	n

functions used in models of deep learning, their associative memory counterparts and the power n which determines the asymptotic behavior of the energy function at $x \rightarrow \infty$. The results of section 4 suggest that for not too large n the speed of learning should improve as n increases. This is consistent with the previous observation that ReLU are faster in training than hyperbolic tangents and logistics [5, 6, 7]. The last row of the table corresponds to rectified polynomials of higher degrees. To the best of our knowledge these activation functions have not been used in neural networks. Our results suggest that for some problems these higher power activation functions should have even better computational properties than the rectified linear units.

6 Discussion and conclusions

What is the relationship between the capacity of the dense associative memory, calculated in section 2, and the neural network with one step update that is used for digit classification? Consider the limit of very large β in (9), so that the hyperbolic tangent is approximately equal to the sign function, as in (4). In the limit of sufficiently large n the network is operating in the prototype regime. The presented image places the initial state of the network close to a local minimum of energy, which corresponds to one of the prototypes. In most cases the one step update of the classification neurons is sufficient to bring this initial state to the nearest local minimum, thus completing the memory recovery. This is true, however, only if the stored patterns are stable and have basins of attraction around them of at least the size of one neuron flip, which is exactly (in the case of random patterns) the condition given by (6). For correlated patterns the maximal number of stored memories might be different from (6), however it still rapidly increases with increase of n . The associative memory with one step update (or the feedforward neural network) is exactly equivalent to the full associative memory with multiple updates in this limit. The calculation with random patterns thus theoretically justifies the expectation of a good performance in the prototype regime.

To summarize, this paper contains three main results. First, it is shown how to use the general framework of associative memory for pattern recognition. Second, a family of models is constructed that can learn representations of the data in terms of features or in terms of prototypes, and that smoothly interpolates between these two extreme regimes by varying the power of interaction vertex. Third, there exists a simple duality between a one step update version of the associative memory model and a feedforward neural network with one layer of hidden units and an unusual activation function. This duality makes it possible to propose a class of activation functions that encourages the network to learn representations of the data with various proportions of features and prototypes. These activation functions can be used in models of deep learning and should be more effective than the standard choices. They allow the networks to train faster. We have also observed an improvement of generalization ability in networks trained with the rectified parabola activation function compared to the ReLU for the case of MNIST. While these ideas were illustrated using the simplest architecture of the neural network with one layer of hidden units, the proposed activation functions can also be used in multilayer architectures. We did not study various regularizations (weight decay, dropout, etc), which can be added to our construction. The performance of the model supplemented with these regularizations, as well as performance on other common benchmarks, will be reported elsewhere.

Appendix A. Details of experiments with MNIST.

The networks were trained using stochastic gradient descent with minibatches of a relatively large size, 100 digits of each class, 1000 digits in total. Training was done for 3000 epochs. Initial weights were generated from a Gaussian distribution $N(-0.3, 0.3)$. Momentum ($0.6 \leq p \leq 0.95$) was used to smooth out oscillations of gradients coming from the individual minibatches. The learning rate was decreasing with time according to

$$\varepsilon(t) = \varepsilon_0 f^t, \quad f = 0.998, \quad (12)$$

where t is the number of epoch. Typical values are $0.01 \leq \varepsilon_0 \leq 0.04$. The weights (memories) were updated after each minibatch according to

$$\begin{aligned} V_I^\mu(t) &= pV_I^\mu(t-1) - \partial_{\xi_I^\mu} C \\ \xi_I^\mu(t) &= \xi_I^\mu(t-1) + \varepsilon \frac{V_I^\mu(t)}{\max_j |V_j^\mu(t)|}, \end{aligned} \quad (13)$$

where t is the number of update, $I = (i, \alpha)$ is an index which unites the visible and the classification units. The proposed update in (13) is normalized so that the largest update of the weights for each hidden unit (memory) is equal to ε . This normalization is equivalent to using different learning rates for each individual memory. It prevents the network from getting stuck on a plateau. All weights were constrained to stay within the $-1 \leq \xi_I^\mu \leq 1$ range. Therefore, if after an update some weights exceeded 1, they were truncated to make them equal to 1 (and similarly for -1). The slope of the function $g(x)$ in (9) is controlled by the effective temperature $\beta = 1/T^n$, which is measured in “neurons” or “pixels”. For large n the temperature can be kept constant throughout the entire training ($500 \leq T \leq 700$). For small n we found useful to start at a high temperature T_i , and then linearly decrease it to the final value T_f during the first 200 epochs ($250 \leq T_i \leq 400$, $30 \leq T_f \leq 100$). The temperature stays constant after that. All the models have $K = 2000$ memories (hidden units).

The MNIST dataset contains 60000 training examples, which were randomly split into 50000 training cases and 10000 validation cases. For each hyperparameter a window of values was selected, such that the error on the validation set after 3000 epochs is less than a certain threshold. After that the entire set of 60000 examples was used to train the network (for 3000 epochs) for various values of the hyperparameters from this optimal window to evaluate the performance on the test set. The validation set was not used for early stopping.

The objective function is given by

$$C = \sum_{\substack{\text{training} \\ \text{examples}}} \sum_{\alpha=1}^{N_c} (c_\alpha - t_\alpha)^{2m}, \quad (14)$$

where t_α is the target output ($t_\alpha = -1$ for the wrong classes and $t_\alpha = +1$ for the correct class). The case $m = 1$ corresponds to the standard quadratic error. For large powers m the function x^{2m} is small for $|x| < 1$ and rapidly grows for $|x| > 1$. Therefore, higher values of m emphasize training examples which produce largest discrepancy with the target output more strongly compared to those examples which are already sufficiently close to the target output. Such emphasis encourages the network to concentrate on correcting mistakes and moving the decision boundary farther away from the barely correct examples rather than on fitting better and better the training examples which have already been easily and correctly classified. Although much of what we discuss is valid for arbitrary value of m , including $m = 1$, we found that higher values of m reduce overfitting and improve generalization at least in the limit of large n . For small n , we used $m = 2, 3, 4$. For $n = 20, 30$, larger values of $m \approx 30$ worked better. We also tried cross-entropy objective function together with softmax output units. The results were worse and are not presented here.

The training can be done both in the associative memory description and in the neural network description. The two are related by the duality of section 5. Below we give the explicit expressions for the update rule (13) for these two methods.

Consider a minibatch of size M . In the associative memory framework one can define two $(N + N_c) \times MN_c$ matrices $U^{\alpha A}$ and $V^{\alpha A}$ (index $A = 1 \dots M$ runs over the training examples of the

minibatch, greek indices $\alpha, \gamma = 1 \dots N_c$ run over classification neurons, index $i = 1 \dots N$ runs over visible neurons, indices $I, J = 1 \dots (N + N_c)$ unite all the neurons, visible and classification).

$$\begin{aligned} U_i^{\alpha A} &= v_i^A & V_i^{\alpha A} &= v_i^A \\ U_\gamma^{\alpha A} &= -1 & V_\gamma^{\alpha A} &= \begin{cases} +1, & \alpha = \gamma \\ -1, & \alpha \neq \gamma \end{cases} \end{aligned}$$

The update rule (9) can then be rewritten as

$$c_\alpha^A = g \left[\beta \left(\sum_{\mu=1}^K F_n(\xi_J^\mu V_J^{\alpha A}) - F_n(\xi_J^\mu U_J^{\alpha A}) \right) \right],$$

where $F_n(x)$ is the rectified polynomial of power n , and summation over index J is assumed. The derivative of the objective function (14) is given by

$$\partial_{\xi_I^\mu} C = (2m\beta n) \sum_{A=1}^M \sum_{\alpha=1}^{N_c} (c_\alpha^A - t_\alpha^A)^{2m-1} \left[1 - (c_\alpha^A)^2 \right] \left[F_{n-1}(\xi_J^\mu V_J^{\alpha A}) V_I^{\alpha A} - F_{n-1}(\xi_J^\mu U_J^{\alpha A}) U_I^{\alpha A} \right]$$

The indices A and α can be united in one tensor product index, so that the two sums can be efficiently calculated using matrix-matrix multiplication.

While this way of training the network is most closely related to the theoretical calculations presented in the main text, it is computationally inefficient. The second dimension of the matrices U and V is N_c times larger than the size of the minibatch. This can become problematic if the classification problem involves many classes. For this reason it is computationally easier to train the dense memory in the dual description, which is more closely related to the conventional methods used in deep learning. In this framework, the minibatch matrix v_i^A has $N \times M$ elements. The update rule is

$$c_\alpha^A = g \left[\beta \sum_{\mu=1}^K \xi_\alpha^\mu f_n(\xi_i^\mu v_i^A) \right],$$

where $f_n(x)$ is a rectified polynomial of power³ n , and summation over the visible index $i = 1 \dots N$ is assumed. The derivatives of the objective function (14) are given by

$$\begin{aligned} \partial_{\xi_i^\mu} C &= (2m\beta n) \sum_{A=1}^M \sum_{\alpha=1}^{N_c} (c_\alpha^A - t_\alpha^A)^{2m-1} \left[1 - (c_\alpha^A)^2 \right] \xi_\alpha^\mu f_{n-1}(\xi_j^\mu v_j^A) v_i^A \\ \partial_{\xi_\alpha^\mu} C &= (2m\beta) \sum_{A=1}^M (c_\alpha^A - t_\alpha^A)^{2m-1} \left[1 - (c_\alpha^A)^2 \right] f_n(\xi_j^\mu v_j^A), \end{aligned}$$

where summation over the visible index j is assumed. These expressions are very similar to the conventional derivatives used in networks with rectified linear activation functions, but they use power activation functions instead. The minibatch training can be efficiently implemented on GPU.

Appendix B. Capacity of Dense associative memory.

In section 2 of the main text a theoretical calculation of the capacity for model (4) was presented in the case of power energy functions. In section 5 an intuitive argument (based on the low energy states of the Hamiltonian) was given arguing that the capacities of the models with power energy functions and rectified polynomial energy functions should be very similar. In this appendix we compare the theoretical results of section 2 with numerical simulations and numerically validate the intuitive argument about low energy states.

A random set of $K = 2000$ binary memory vectors was generated in the model with $N = 100$ neurons. A collection of 10000 random initial configurations of binary spins were evolved according to (4) until convergence. The quality of memory recovery was measured by the overlap between

³One should remember that the energy function of power n is dual to the activation function of power $n - 1$. Here, for the sake of notations, we describe the training procedure for general n .

the final configuration of spins ($\sigma_i = \sigma_i^{(t \rightarrow \infty)}$) and the closest memory, $\max_{\mu} \left(\sum_{i=1}^N \xi_i^{\mu} \sigma_i \right)$. If the recovery is perfect, this quantity is equal to N ; if some of the spins failed to match a memory vector, this quantity is smaller than N . In Fig. 4 the histograms of the overlaps are shown for $n = 2, 3, 4$ in case of power and rectified polynomial energy functions. For $n = 2, 3$ the number of memories ($K = 2000$) places the model above the capacity (according to (6), $K_{\text{no errors}}^{\text{max}} \approx 11$ for $n = 2$ and $K_{\text{no errors}}^{\text{max}} \approx 360$ for $n = 3$). Thus, the model is unable to reconstruct the memories. For $n = 4$, the number of memories is below the capacity ($K_{\text{no errors}}^{\text{max}} \approx 7240$), thus the distribution sharply peaks at perfect recovery. For $n \geq 5$ all 10000 samples converge to one of the memories. Qualitatively, this behavior is demonstrated by both power models and rectified models.

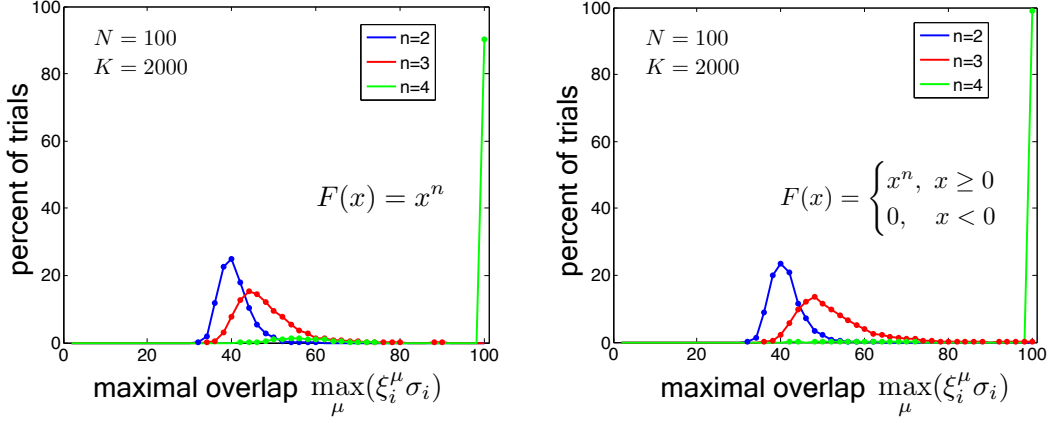


Figure 4: The histograms of overlaps for models with $n = 2, 3, 4$ with power energy functions (left) and rectified polynomial energy functions (right). Each histogram has 10000 samples in it.

A family of models with $50 \leq N \leq 200$ and $50 \leq K \leq 1500$ was studied. For each combination of N and K a set of binary memory vectors was generated to make a model of associative memory. After that 1000 random binary initial conditions were evolved according to (4) until convergence. $K_{1/2}$ is the number of memories when half (500) of these samples perfectly converge to one of the memories. In Fig. 5 the $K_{1/2}$ dependence of N is shown for the power and the rectified models with $n = 3$. The solid curve is given by Eq.(6). The results of numerical simulations for the case of power activation functions are consistent with the theoretical calculation (6). The results for the rectified polynomials are a little bit above the theoretical curve, but show similar non-linear behavior.

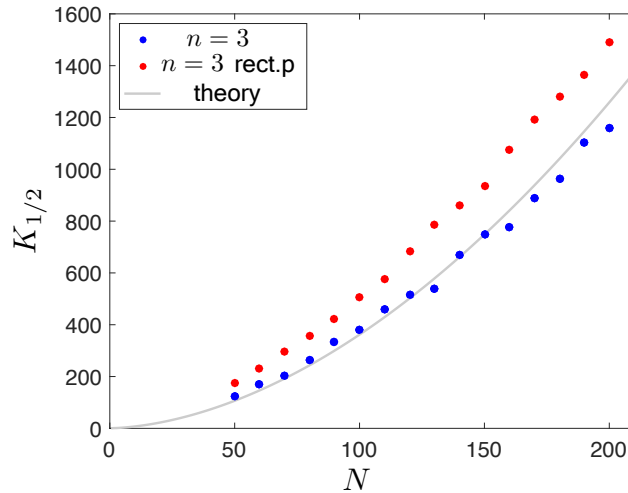


Figure 5: Scaling behavior of the capacity vs. the number of neurons for $n = 3$ with power and rectified polynomial energy functions. Solid curve is the theoretical result (6).

Acknowledgments

We thank B. Chazelle, D. Huse, A. Levine, M. Mitchell, R. Monasson, L. Peliti, D. Raskovalov, B. Xue, and all the members of the Simons Center for Systems Biology at IAS for useful discussions. We especially thank Y. Roudi for pointing out the reference [13] to us. The work of DK is supported by Charles L. Brown membership at IAS.

References

- [1] Hopfield, J.J., 1982. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the national academy of sciences*, 79(8), pp.2554-2558.
- [2] LeCun, Y., Chopra, S., Hadsell, R., Ranzato, M. and Huang, F., 2006. A tutorial on energy-based learning. *Predicting structured data*, 1, p.0.
- [3] Hinton, G.E., Osindero, S. and Teh, Y.W., 2006. A fast learning algorithm for deep belief nets. *Neural computation*, 18(7), pp.1527-1554.
- [4] Hinton, G.E. and Salakhutdinov, R.R., 2006. Reducing the dimensionality of data with neural networks. *Science*, 313(5786), pp.504-507.
- [5] Nair, V. and Hinton, G.E., 2010. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML-10)* (pp. 807-814).
- [6] Glorot, X., Bordes, A. and Bengio, Y., 2011. Deep sparse rectifier neural networks. In *International Conference on Artificial Intelligence and Statistics* (pp. 315-323).
- [7] Krizhevsky, A., Sutskever, I. and Hinton, G.E., 2012. ImageNet classification with deep convolutional neural networks. In *Advances in neural information processing systems* (pp. 1097-1105).
- [8] Amit, D.J., Gutfreund, H. and Sompolinsky, H., 1985. Storing infinite numbers of patterns in a spin-glass model of neural networks. *Physical Review Letters*, 55(14), p.1530.
- [9] McEliece, R.J., Posner, E.C., Rodemich, E.R. and Venkatesh, S.S., 1987. The capacity of the Hopfield associative memory. *Information Theory, IEEE Transactions on*, 33(4), pp.461-482.
- [10] Kanter, I. and Sompolinsky, H., 1987. Associative recall of memory without errors. *Physical Review A*, 35(1), p.380.
- [11] Chen, H.H., Lee, Y.C., Sun, G.Z., Lee, H.Y., Maxwell, T. and Giles, C.L., 1986. High order correlation model for associative memory. In *Neural Networks for Computing* (Vol. 151, No. 1, pp. 86-99). AIP Publishing.
- [12] Psaltis, D. and Park, C.H., 1986. Nonlinear discriminant functions and associative memories. In *Neural networks for computing* (Vol. 151, No. 1, pp. 370-375). AIP Publishing.
- [13] Baldi, P. and Venkatesh, S.S., 1987. Number of stable points for spin-glasses and neural networks of higher orders. *Physical Review Letters*, 58(9), p.913.
- [14] Gardner, E., 1987. Multiconnected neural network models. *Journal of Physics A: Mathematical and General*, 20(11), p.3453.
- [15] Abbott, L.F. and Arian, Y., 1987. Storage capacity of generalized networks. *Physical Review A*, 36(10), p.5091.
- [16] Horn, D. and Usher, M., 1988. Capacities of multiconnected memory models. *Journal de Physique*, 49(3), pp.389-395.
- [17] Minsky, M. and Papert, S., 1969. *Perceptron: an introduction to computational geometry*. The MIT Press, Cambridge, expanded edition, 19(88), p.2.
- [18] Simard, P.Y., Steinkraus, D. and Platt, J.C., 2003, August. Best practices for convolutional neural networks applied to visual document analysis. In *null* (p. 958). IEEE.
- [19] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R., 2014. Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1), pp.1929-1958.
- [20] Wan, L., Zeiler, M., Zhang, S., LeCun, Y. and Fergus, R., 2013. Regularization of neural networks using dropconnect. In *Proceedings of the 30th International Conference on Machine Learning (ICML-13)* (pp. 1058-1066).
- [21] Goodfellow, I.J., Shlens, J. and Szegedy, C., 2014. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*.
- [22] Rasmus, A., Berglund, M., Honkala, M., Valpola, H. and Raiko, T., 2015. Semi-supervised learning with ladder networks. In *Advances in Neural Information Processing Systems* (pp. 3546-3554).